

Připomenutí principů návrhu hardware pomocí VHDL, simulace a verifikace hardware

INP - cvičení 1

Zdeněk Vašíček, Vojtěch Mrázek, 2021
{vasicek, mrazek}@fit.vutbr.cz

VHDL – základní pojmy

VHDL

- VHDL je jazyk umožňující popis hardware běžně používaný v elektrotechnickém průmyslu na celém světě
- Dovoluje popisovat číslicové, ale také analogové a smíšené systémy (viz rozšíření VHDL-AMS)
- VHDL je zkratkou VHSIC HDL (Very High Speed Integrated Circuit Hardware Description Language)
- VHDL prochází standardizací (poslední revize je z roku 2009 a označuje se VHDL 1076-2008)

HDL jazyky a jejich využití

- V praxi se HDL jazyky běžně používají pro návrh (prototypování) procesorů a obecně hardware
 - např. **INTEL** (CPU), **NVIDIA** (GPU), **ARM** (CPU), ...
- Předností HDL je přenositelnost kódu napříč platformami
 - využíváno při **prototypování ASIC obvodů** pomocí FPGA z důvodu extrémně drahé výroby čipů (viz např. **ARM** CPU); podpora od Cadence, Synopsys, S2C (SMIT)

ARM in the World of FPGA-Based Prototyping

<http://goo.gl/SOGlwR>

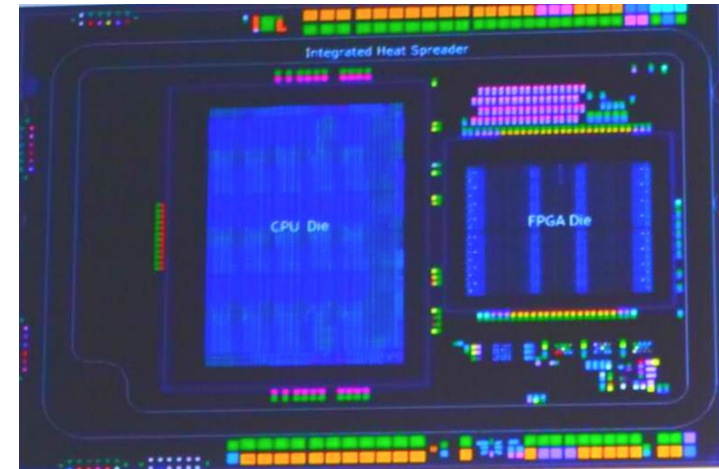
- Každý komerční nástroj podporuje minimálně dva HDL jazyky
 - VHDL, Verilog

Cesta k energeticky udržitelnému škálování

- Nabídnout systém integrující FPGA v CPU (Intel 2016, 14nm Xeon)

“Intel officials have said they expect FPGAs will be used in as much as 30 percent of data center servers by 2020.”

“Bryant in China said the combination of the FPGAs and Xeon E5 chips will drive a 70 percent improvement in performance-per-watt to systems.”



Intel Begins Shipping Xeon Chips With FPGA Accelerators

<http://goo.gl/8pEHw0>

- Zákazníci: Amazon Web Services, Facebook, Microsoft, Google, Baidu
- Výhodu získají ti, kteří budou umět takové systémy programovat (HDL, OpenCL).

Amazon AWS

- K FPGA se lze dnes dostat poměrně jednoduše
 - např. Amazon poskytuje instanci Amazon EC2 F1 vybavenou obvody FPGA a potřebnou infrastrukturou pro přenos dat mezi CPU / pamětí / FPGA



Amazon EC2 F1 Instances

Run Customizable FPGAs in the AWS Cloud

FPGA Instances - Current Generation

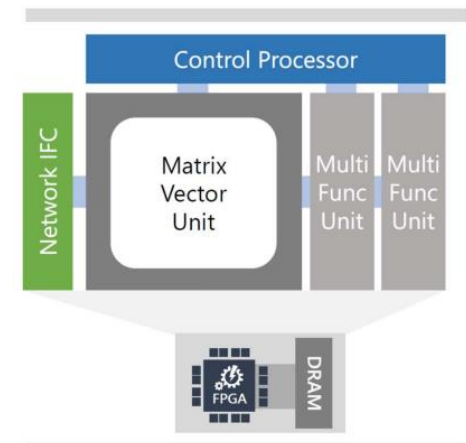
Name	FPGAs	vCPUs	Instance Memory (GiB)	SSD Storage (GB)	Enhanced Networking	EBS Optimized	On-Demand Price/hr*	1-yr Reserved Instance Effective Hourly*	3-yr Reserved Instance Effective Hourly*
f1.2xlarge	1	8	122	470	Yes	Yes	\$1.65	\$1.06	\$0.76
f1.4xlarge	2	16	244	940	Yes	Yes	\$3.30	\$2.12	\$1.52
f1.16xlarge	8	64	976	4 x 940	Yes	Yes	\$13.20	\$8.50	\$6.10

Současné trendy – strojové učení

- V roce 2015 se podařilo překonat schopnosti člověka v rozpoznávání obrazů (ImageNet database).
 - “ A number of companies – both large companies [Google, Microsoft/Intel] and a slew of startups [60+] – are building hardened DNN (Deep Neural Network) processing units”.

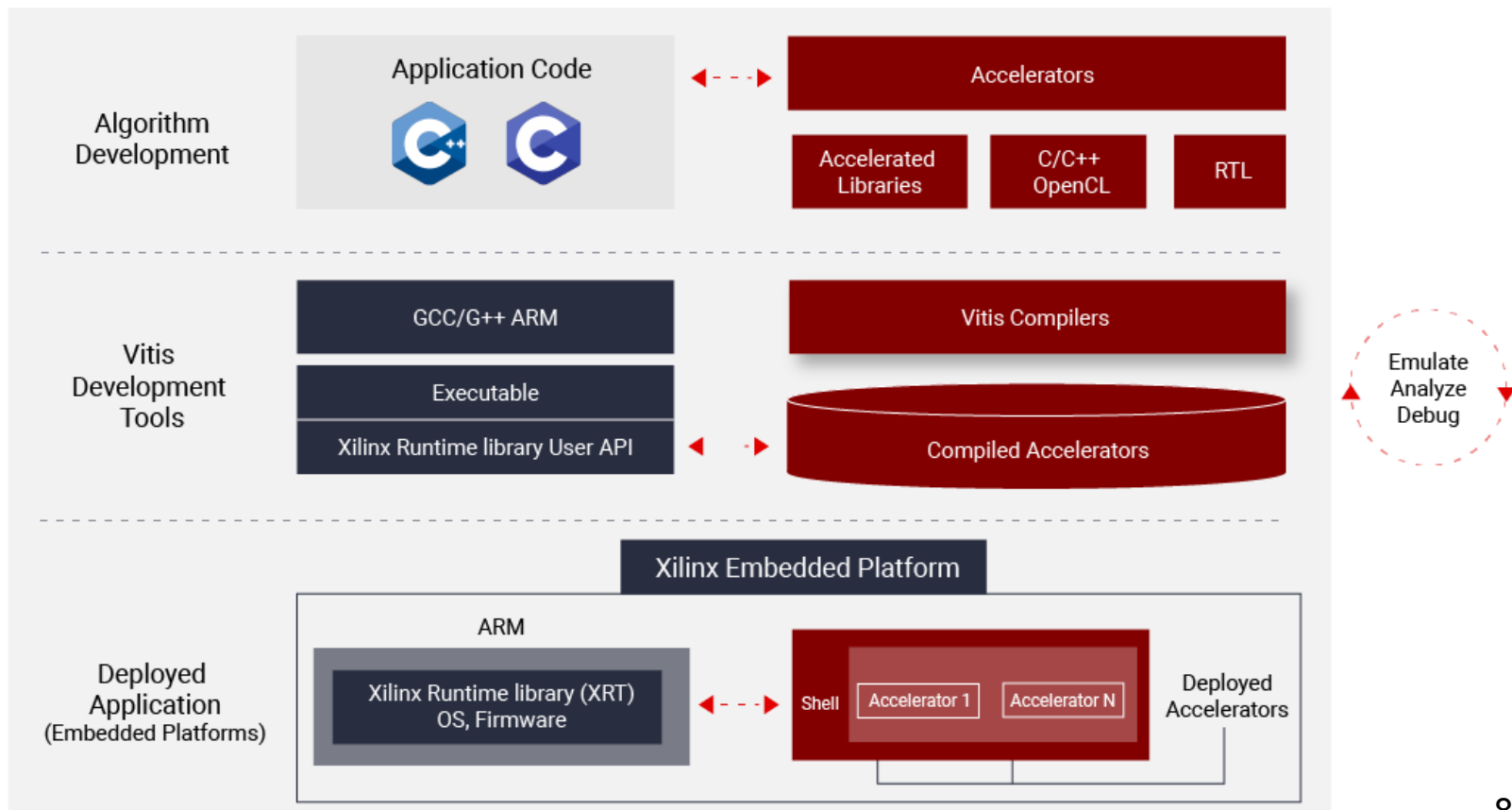
Microsoft unveils Project Brainwave for real-time AI
<https://goo.gl/GHS85H>

- Microsoft BrainWave Project (2017)
 - FPGA Stratix 10, 14nm, distribuované BRAM
 - propustnost desítky TB/sec



Strojové učení – nástup algoritmů pro HLS

- V roce 2019 byla představena nová rodina čipů Xilinx Vitis, která integruje speciální jádra vhodná pro strojové učení (např. konvoluce)
- Neustále však musíme rozumět tomu, co se děje „pod pokličkou“.



Návrh hardware v praxi

Role VHDL

Zdrojový kód (VHDL, Verilog, ...)

```
library IEEE;  
use IEEE.std_logic_1164.all;  
  
entity blk4 is  
  port (  
    i1: in STD_LOGIC;  
    i2: in STD_LOGIC;  
    i3: in STD_LOGIC;  
    i4: in STD_LOGIC;  
    o1: out STD_LOGIC  
  );  
end blk4;  
  
architecture struc of blk4 is  
begin  
  o1 <= (i1 or i2) and (i3 and i4);  
end struc;
```

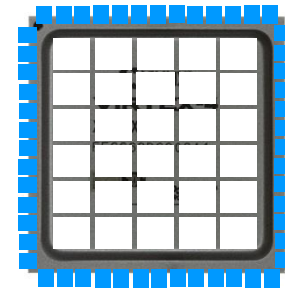
syntéza,
place&route, map

Virtex-UltraScale (největší FPGA):

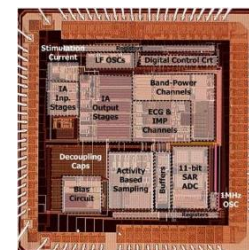
4.4 miliony buněk
123MB konfigurace

Spartan-3 (FITkit):

1728 buněk
54kB konfigurace



FPGA



ASIC

HDL jazyky a klasické programovací jazyky

- Odlišnosti od tradičních jazyků:
 - umožňuje modelovat souběžné děje
(systém popsán pomocí paralelně pracujících komponent).
 - specifické datové objekty a typy
(signály, sběrnice, vícehodnotová logika, fyzikální datové typy)
 - koncept času, schopnost modelovat elektrické vlastnosti
(zpoždění, parametry hradel – setup, hold apod.)
- HDL je možné použít pro
 - simulaci (digital i mixed-signal, modely, validace, verifikace)
 - syntézu (popis hardware – FPGA i ASIC)
 - specifikaci (obecný prog. jazyk)

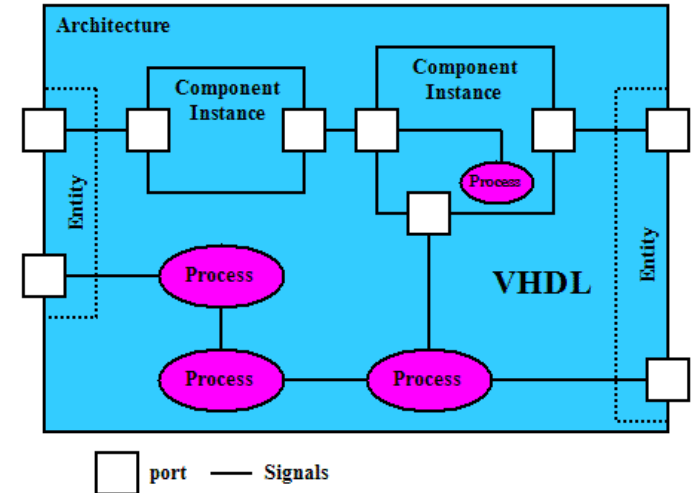
Záludnosti VHDL

- Jazyk VHDL byl navržen s ohledem na efektivitu simulace chování elektronických systémů
 - nepochopení konstrukce může způsobit odlišné chování od simulace (např. sensitivity list)
- VHDL obsahuje konstrukce usnadňující tvorbu simulačních prostředí
 - ne každá konstrukce je syntetizovatelná (např. while)
- Nenechávejte si řešení projektu na poslední chvíli!

Stručné připomenutí principů návrhu HW pomocí VHDL

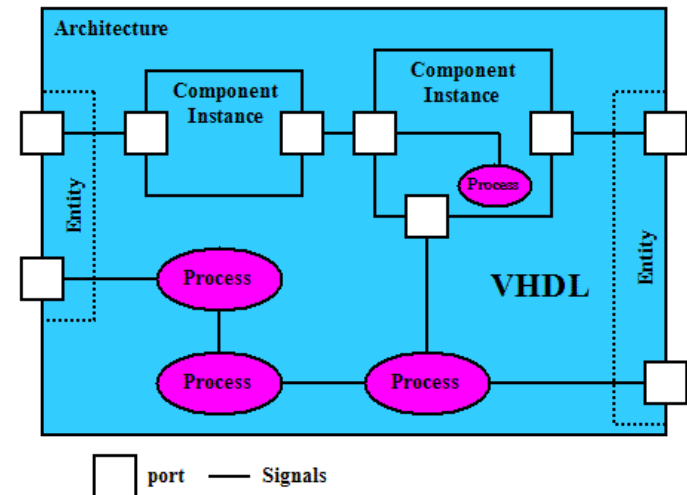
Princip modelování HW systémů pomocí VHDL

- Modelovaný systém je chápán jako množina vzájemně komunikujících komponent.
- Komunikace probíhá pomocí signálů (speciální k tomuto účelu vyhrazený datový objekt).
- Kompletní systém je typicky modelován jako hierarchie komponent (návrh shora dolů).



Způsob modelování komponent

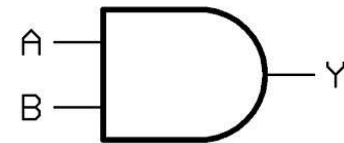
- Ve VHDL je každá **komponenta modelována dvojicí**
 - **entita** (jak vypadá rozhraní)
 - **architektura** (jak se komponenta chová)
- Jednotlivé části popisu jsou oddělitelné
 - jedna entita může mít několik různých architektur (např. jednu pro simulaci, jinou pro syntézu)



Entita

- Entita popisuje rozhraní mezi komponentou a jejím okolím
- Rozhraní komponenty definuje
 - seznam signálů rozhraní (volitelně)
 - seznam generických parametrů sloužících k předávání dalších parametrů do entity (volitelně)

```
library IEEE;  
use IEEE.std_logic_1164.all;  
  
-- entity  
entity ANDGATE is  
    port (  
        A: in std_logic;  
        B: in std_logic;  
        Y: out std_logic  
    );  
end ANDGATE;
```



Signály entity

- Signály rozhraní mohou být v jednom ze čtyř režimů
 - **IN** (vstup, hodnota nemůže být uvnitř komponenty modifikována)
 - **OUT** (výstup, hodnota nemůže být uvnitř komponenty čtena)
 - BUFFER (výstup, hodnotu však lze uvnitř komponenty číst)
 - INOUT (vstup-výstup, obousměrný port, sběrnice)

Architektura

- Architektura popisuje chování a/nebo strukturu komponenty a je vždy svázána s entitou.
- Architektura obsahuje
 - Deklarační část učenou k deklaraci signálů, konstant, typů nebo funkcí použitých uvnitř architektury.
 - Sekci paralelních příkazů obsahující instance komponent a procesy propojené pomocí signálů. **Nezáleží na pořadí příkazů!**

```
library IEEE;  
use IEEE.std_logic_1164.all;  
  
-- entity  
entity ANDGATE is  
    port (  
        A: in std_logic;  
        B: in std_logic;  
        Y: out std_logic  
    );  
end ANDGATE;
```

```
architecture RTL of ANDGATE is  
    --deklarace  
begin  
  
    Y <= A and B;  
  
end RTL;
```

Možnosti popisu chování

- V rámci sekce paralelních příkazů lze použít
 - data-flow popis
 - specifikace funkce pomocí logických výrazů
 - lze přepsat pomocí konstrukce process
 - strukturní popis
 - instancování komponent a jejich propojení pomocí signálů
 - behaviorální popis
 - popis algoritmů a funkčních bloků pomocí procesů komunikujících prostřednictvím signálů
 - ne vše co lze zapsat behaviorálně je syntetizovatelné!
- Jednotlivé přístupy lze libovolně kombinovat

Konstrukce process

Základní prvek behaviorálního popisu

- Konstrukce process vymezuje úsek kódu, jehož příkazy jsou vyhodnocovány v průběhu **simulace** sekvenčně.

Pořadí příkazů je podstatné!

```
[label:] process [(sensitivity list)]  
    deklarční část  
begin  
    část sekvenčních příkazů  
end process [label];
```

- Process obsahuje
 - **Sensitivity list** (nepovinný) – seznam aktivátorů procesu
 - **Deklarční část** (nepovinná) – je vyhrazena pro deklaraci proměnných, konstant nebo typů použitých uvnitř procesu.
 - **Část sekvenčních příkazů** – popisují chování komponenty nebo její části.

Záludnosti konstrukce process

Sensitivity list

- Sensitivity list

- seznam signálů, jejichž změna jediné hodnoty má za následek provedení příkazů uvedených uvnitř procesu
- informuje simulátor, kdy je nutné přepočítat signály, které jsou tímto procesem řízeny
- **slouží pouze k zefektivnění simulace**, syntézní nástroje jej ignorují neboť provádějí analýzu kódu

- Časté chyby

1. signál není omylem uveden v sensitivity listu
2. sensitivity list obsahuje nadbytečné signály tak, aby simulace pracovala korektně

- Důsledek těchto chyb

- v praxi se obvod chová jinak než v rámci simulace

Jednoduchá pravidla pro tvorbu sensitivity listu

1. Kombinační logika

- nemá hodinový signál
- výstup musí reagovat na změnu kteréhokoliv z použitých signálů
- sensitivity list obsahuje všechny signály na pravé straně přiřazení a všechny signály použité v podmínce či proceduře

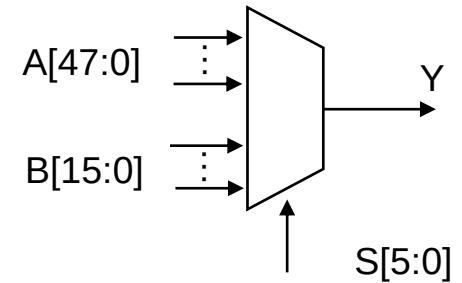
2. Sekvenční logika

- pracuje pouze na hraně hodinového signálu
- výstup se mění pouze je-li detekována hrana hodinového signálu (modeluje se podmínkou testující atribut 'EVENT')
- sensitivity list obsahuje zpravidla pouze hodinový signál CLK, případně **asynchronní** nulování RESET

Kombinační prvek pomocí konstrukce process

Multiplexor varianta 64-1

```
entity mux64 is
    port ( A: in std_logic_vector (47 downto 0);
          B: in std_logic_vector (15 downto 0);
          S: in std_logic_vector (5 downto 0);
          Y: out std_logic);
end mux64;
```



Multiplexor pomocí konstrukce for-loop

```
architecture beh of mux64 is begin
    process (A, B, S)
    begin
        for i in 0 to 63 loop
            if (i = conv_integer(S)) then
                if (i < 48) then
                    Y <= A(i);
                else
                    Y <= B(i-48);
                end if;
            end if;
        end loop;
    end process;
end architecture;
```

Multiplexor pomocí vektorové aritmetiky

```
architecture beh2 of mux64 is
begin
    process (A, B, S)
        variable muxin : std_logic_vector(63
            downto 0);
    begin
        muxin := B & A;
        Y <= muxin(conv_integer(S));
    end process;
end architecture;
```

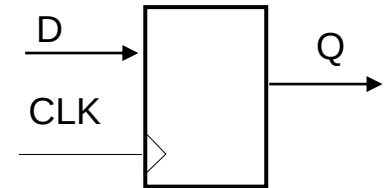
Zavedením proměnné odpadá
nutnost používat přetypování,
navíc $T'(B \& A)(\dots)$ není
syntakticky validní

Sekvenční prvek pomocí konstrukce process

klopný obvod typu D

Klopný obvod typu D je synchronní obvod, který při každé vzestupné hodinové hraně kopíruje stav vstupu D na výstup Q

```
library IEEE;
use ieee.std_logic_1164.all;
entity reg is
    port (
        CLK : IN std_logic;
        D    : IN std_logic;
        Q    : OUT std_logic
    );
end reg;
```



```
architecture behavioral of reg is
begin

    reg: process (CLK)
    begin
        if CLK'event and (CLK = '1') then
            -- if rising_edge(CLK) then
                Q <= D;
            end if;
        end process;

end architecture;
```

proces se aktivuje s každou změnou CLK, **nikoliv změnou D**

přiřazení je vykonáno pouze tehdy, došlo-li k události na CLK a CLK je v log. 1 (tzn. byla detekována vzestupná hrana)

Záludnosti konstrukce process

Příkaz přiřazení a sémantika procesu

- Co je nutné vědět o příkazech přiřazení
 - Přiřazení hodnoty signálu ($\leq$) není vykonáno ihned, ale až po ukončení procesu (přirozené nebo voláním příkazu wait). Tato sémantika umožňuje modelovat paralelismus.
 - Přiřazení hodnoty proměnné ($:=$) je vykonáno ihned. Zavedeno proto, aby bylo možné zkracovat zápisy v podmínkách a pravých stranách přiřazení (tzn. používat **dočasné** proměnné).
 - Ačkoliv jsou interpretovány příkazy sekvenčně, provádějí se jednotlivá přiřazení hodnot signálům paralelně.
 - Jeden konkrétní **signál nelze ovládat z více procesů** současně (nejedná-li se o sběrnici).
 - Proměnná je inicializována pouze při prvním spuštění procesu!

Process

proměnné versus signály

```
architecture VAR of EXAMPLE is
  signal CLK, RESULT: integer :=0;
begin

  process(CLK)
    variable a1: integer :=1;
    variable a2: integer :=2;
    variable a3: integer :=3;
  begin
    if CLK'event and CLK='1' then
      a1 := a2;
      a2 := a1 + a3;
      a3 := a2;
      RESULT <= a1 + a2 + a3;
    end if;
  end process;

end architecture;
```

RESULT: 12, 25, ...

```
architecture SIG of EXAMPLE is
  signal CLK, RESULT: integer := 0;
  signal a1: integer :=1;
  signal a2: integer :=2;
  signal a3: integer :=3;
begin

  process(CLK)
  begin
    if CLK'event and CLK='1' then
      a1 <= a2;
      a2 <= a1 + a3;
      a3 <= a2;
      RESULT <= a1 + a2 + a3;
    end if;
  end process;

end architecture;
```

RESULT: 6, 8, ...

1. Proměnné jsou inicializovány pouze při elaboraci! (obdoba static v C)
2. Hodnota proměnné je přepsána bezprostředně po vykonání přiřazení

Process

proměnné versus signály

```
architecture VAR of EXAMPLE is
    signal CLK, RESULT: integer :=0;
begin

    process(CLK)
        variable a1: integer :=1;
        variable a2: integer :=2;
        variable a3: integer :=3;
    begin
        if CLK'event and CLK='1' then
            RESULT <= a1 + a2 + a3;
            a1 := a2;
            a2 := a1 + a3;
            a3 := a2;
        end if;
    end process;

end architecture;
```

RESULT: ?

```
architecture SIG of EXAMPLE is
    signal CLK, RESULT: integer := 0;
    signal a1: integer :=1;
    signal a2: integer :=2;
    signal a3: integer :=3;
begin

    process(CLK)
    begin
        if CLK'event and CLK='1' then
            RESULT <= a1 + a2 + a3;
            a1 <= a2;
            a2 <= a1 + a3;
            a3 <= a2;
        end if;
    end process;

end architecture;
```

RESULT: ?

Process

proměnné versus signály

```
architecture VAR of EXAMPLE is
  signal CLK, RESULT: integer :=0;
begin

  process(CLK)
    variable a2: integer :=2;
    variable a3: integer :=3;
  begin
    if CLK'event and CLK='1' then
      RESULT <= 3*a2 + 2*a3;
      a2 := a2 + a3;
      a3 := a2;
    end if;
  end process;

end architecture;
```

RESULT: ?

```
architecture SIG of EXAMPLE is
  signal CLK, RESULT: integer := 0;
  signal a2: integer :=2;
  signal a3: integer :=3;
begin

  process(CLK)
  begin
    if CLK'event and CLK='1' then
      RESULT <= 3*a2 + 2*a3;
      a2 <= a2 + a3;
      a3 <= a2;
    end if;
  end process;

end architecture;
```

RESULT: ?

Process a sekvenční zpracování příkazů

Jakou bude mít hodnotu signál CNT a LED, pokud bude CNT "1110" a přijde vzestupná hrana hodinového signálu.

```
process(CLK)
begin
  if (CLK'event) and (CLK='1') then

    if (cnt = "1110") then
      cnt <= "0000";
    else
      cnt <= cnt + 1;
    end if;

    led <= '0';
    if (cnt = "1110") then
      led <= '1';
    end if;

  end if;
end process;
```

Process a sekvenční zpracování příkazů

použití pravidla jeden process = jedna komponenta

Jakou bude mít hodnotu signál CNT a LED nyní?

```
process(CLK)
begin
  if (CLK'event) and (CLK='1') then
    if (cnt = "1110") then
      cnt <= "0000";
    else
      cnt <= cnt + 1;
    end if;
  end if;
end process;
```

```
process(CLK)
begin
  if (CLK'event) and (CLK='1') then
    led <= '0';
    if (cnt = "1110") then
      led <= '1';
    end if;
  end if;
end process;
```

Jak se vyhnout problémům

aneb pravidla pro popis HW pomocí VHDL

- Každý systém lze rozložit na kombinační a sekvenční část, tvorba HW tak spočívá ve vhodném spojování základních stavebních bloků (tzn. **syntetizovatelných šablon**)
 - V Xilinx ISE viz menu *Edit* položka *Language Templates* a dále **VHDL > Synthesis Constructs > Coding Examples**
 - Přehled popisu základních komponent lze nalézt zde:
http://merlin.fit.vutbr.cz/FITkit/docs/navody/synth_templates.html
<https://www.fit.vutbr.cz/study/courses/IVH/private/>
- Návrh je dobré začít **blokovým schematem**, následný přepis do VHDL je rutinní záležitost
- V rámci **jednoho procesu** je vhodné popisovat pouze **jedinou komponentu**, vyhneme se problémům s chybami v konstrukcích.
- **Nepoužívat proměnnou k uchování stavu** mezi hodinovými takty.

Chybné konstrukce

- Jeden konkrétní **signál nelze ovládat z více procesů**

```
architecture BAD of EXAMPLE is
    signal led : std_logic;
begin

    process(CLK)
    begin
        if (CLK'event) and (CLK='1') then
            if (...) then
                led <= '1'
            end if;
        end if;
    end process;

    process(CLK)
    begin
        if (CLK'event) and (CLK='1') then
            if (...) then
                led <= '1';
            end if;
        end if;
    end process;
```



```
architecture VALID of EXAMPLE is
    signal led : std_logic;
begin

    process(CLK)
    begin
        if (CLK'event) and (CLK='1') then
            if (...) or (...) then
                led <= '1'
            end if;
        end if;
    end process;
```

Chybné konstrukce

- Špatný popis sekvenčního prvku s asynchronním nulováním.
Asynchronní nulování musí mít vždy nejvyšší prioritu.

```
architecture BAD of EXAMPLE is
    signal led : std_logic;
begin

    process(RESET, CLK)
    begin
        if (RESET='1') then
            led <= '0';
        end if;
        if (CLK'event) and (CLK='1') then
            if (...) then
                led <= '1'
            end if;
        end if;
    end process;
```



```
architecture VALID of EXAMPLE is
    signal led : std_logic;
begin

    process(RESET, CLK)
    begin
        if (RESET='1') then
            led <= '0';
        elsif (CLK'event) and (CLK='1') then
            if (...) then
                led <= '1'
            end if;
        end if;
    end process;
```

Chybné konstrukce

- Špatný popis sekvenčního prvku s asynchronním nulováním.
Asynchronní nulování musí mít vždy nejvyšší prioritu.

```
architecture BAD of EXAMPLE is
    signal led : std_logic;
begin

    process(RESET, CLK)
    begin
        if (CLK'event) and (CLK='1') then
            if (...) then
                led <= '1'
            end if;
            end if;
            if (RESET='1') then
                led <= '0';
            end if;
        end process;
```



```
architecture VALID of EXAMPLE is
    signal led : std_logic;
begin

    process(RESET, CLK)
    begin
        if (RESET='1') then
            led <= '0';
        elsif (CLK'event) and (CLK='1') then
            if (...) then
                led <= '1'
            end if;
        end if;
    end process;
```

Chybné konstrukce

- Špatný popis sekvenčního prvku s asynchronním nulováním
Asynchronní nulování nesmí nikdy reagovat na synchronní událost

```
architecture BAD of EXAMPLE is
    signal led : std_logic;
begin

    process(RESET, CLK)
    begin
        if rising_edge(RESET) then
            led <= '0';
        end if;
        if falling_edge(RESET) and/or if
            (CLK'event) and (CLK='1') then
            if (...) then
                led <= '1'
            end if;
        end if;
    end if;

end process;
```



```
architecture VALID of EXAMPLE is
    signal led : std_logic;
begin

    process(RESET, CLK)
    begin
        if (RESET='1') then
            led <= '0';
        elsif (CLK'event) and (CLK='1') then
            if (...) then
                led <= '1'
            end if;
        end if;
    end process;
```

Nevhodné konstrukce

- Nevhodný popis sekvenčního chování, který vede na zhoršené parametry výsledného hardware (hodinový signál je veden datovou cestou)

```
architecture BAD of EXAMPLE is
    signal clk_slow : std_logic := '0';
begin

    process(CLK)
    begin
        if (CLK'event) and (CLK='1') then
            if (...) then
                clk_slow <= not clk_slow;
            end if;
        end if;
    end process;

    process(clk_slow)
    begin
        if (clk_slow'event) and (clk_slow = '1')
        then
            ...
        end if;
    end process;
```



```
architecture VALID of EXAMPLE is
    signal clken : std_logic := '0';
begin

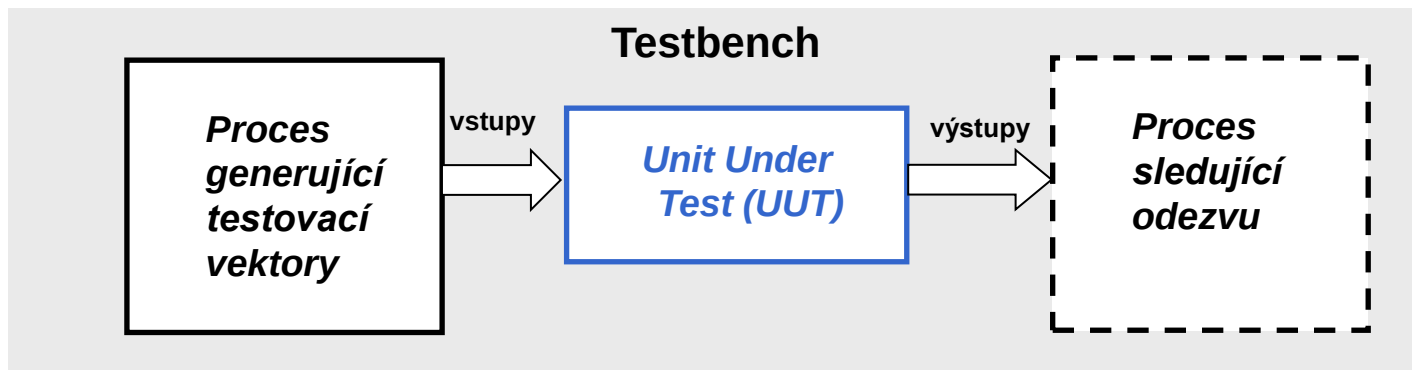
    process(CLK)
    begin
        if (CLK'event) and (CLK='1') then
            clken <= '0';
            if (...) then clken <= '1'; end if;
        end if;
    end process;

    process(CLK)
    begin
        if (CLK'event) and (CLK='1') then
            if (clken = '1') then
                ...
            end if;
        end if;
    end process;
```

Verifikace a simulace pomocí nástroje Xilinx ISE Simulator

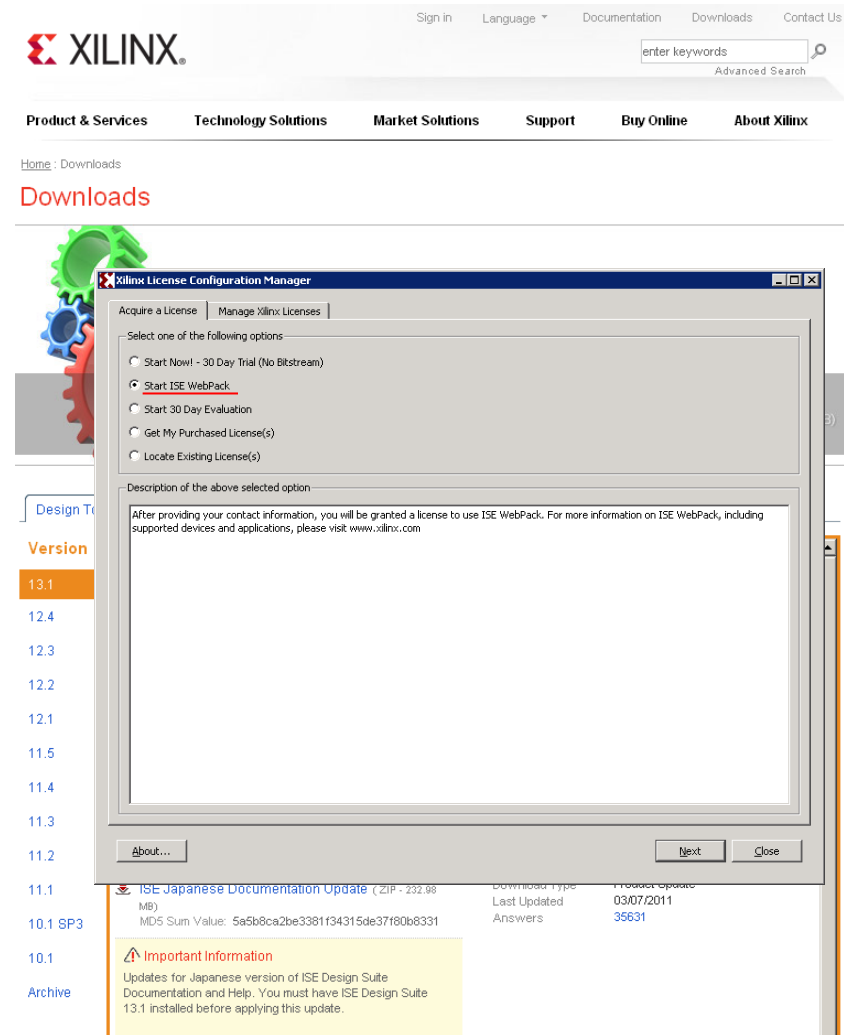
Ověření funkčnosti - testbench

- Testbench je obvod, který aplikuje testovací vektory na vstupy testovaného obvodu a verifikuje získanou odezvu (není nezbytně nutné).
- Výsledkem může být časový diagram (waveform) nebo textový/binární soubor.
- Ve VHDL lze napsat jednoduchý i komplexní testbench (snadná přenositelnost).



Nástroje pro simulaci VHDL

- Pro simulaci lze použít
 - ModelSIM (vyžaduje licenci)
 - Xilinx ISE Simulator (ISIM) (součást volně dostupného balíku Xilinx ISE Webpack)
- SW je k dispozici ve formě image pro VirtualBox dostupného na stránkách FITkitu (<https://merlin.fit.vutbr.cz/FITkit/private/web/download.php>)
- Alternativně lze pracovat přímo na stroji fitkit-build (z Windows pomocí putty+Xming apod.)



<http://www.xilinx.com/support/download/index.htm>

Xilinx ISE Simulator

Součásti simulátoru Xilinx ISE Simulator (ISIM)

- *vhcomp.exe*, *vlogcomp.exe* – parser VHDL, Verilog
- *fuse.exe* – HDL elaborace, linker
přeloží a slinkuje dohromady jednotlivé jednotky VHDL
výstupem je simulátor ve spustitelné podobě (např. *testbench.exe*)
- *testbench.exe* – spustitelný simulátor událostně řízené simulace
- *isimgui.exe* – uživatelské rozhraní ISIM interagující se simulátorem

Práce se simulátorem:

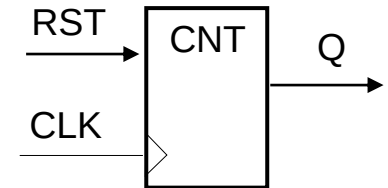
- Celým procesem simulace lze projít z GUI (Xilinx ISE) i příkazové řádky
- Simulaci lze automatizovat pomocí TCL skriptů

Příklad: 8-bitový binární čítač (cntr.vhd)

Synchronní čítač - obvod, který při každé vzestupné (nebo sestupné) hodinové hraně inkrementuje hodnotu na výstupu Q

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity cntr is
    port (
        CLK : IN std_logic;
        RST : IN std_logic;
        Q    : BUFFER std_logic_vector(7 downto 0)
    );
end cntr;
```

```
architecture behavioral of cntr is
begin
    citac: process (RST, CLK)
    begin
        if RST='1' then
            Q <= (others => '0');
        elsif CLK'event and (CLK = '1') then
            -- elsif (CLK = '1') then -- chybný zápis
            Q <= Q + 1;
        end if;
    end process;
end architecture;
```



Příklad: 8-bitový binární čítač

testbench pro čítač (tb.vhd)

```
library ieee; use ieee.std_logic_1164.all;
entity citac_tb is
end citac_tb;

architecture behavior of citac_tb is
    --vstupy
    signal clk : std_logic := '0';
    signal rst : std_logic := '1';
    --vystupy
    signal q : std_logic_vector(7 downto 0);
begin
    -- instance uut (unit under test)
    uut: entity work.cntn port map (
        CLK => clk, RST => rst, Q => q
    );

    clk <= not clk after 5 ns;

    process
    begin
        wait for 100 ns; wait until clk='0';
        rst <= '0';
        wait until clk='1'; wait for 1 ns;
        assert q = "00000001" report "Chyba 1";
        wait until clk='1'; wait for 1 ns;
        assert q = "00000010" report "Chyba 2";
        ...
        wait;
    end process;
end;
```

Typické části

- deklarace signálů potřebných pro simulaci
- instance simulované komponenty (pomocí entity nebo komponenty)
- generátor hodinového signálu
- proces řídicí signál rst a sledující odezvu

Xilinx ISIM tutorial

založení nového projektu

- Spustit Xilinx ISE Design Suite
- Založení nového projektu
 - *File > New Project*
 - zvolit konkrétní typ FPGA, např. XC3S50-PQ208 (Spartan-3 na FITkitu)

The screenshot shows the 'New Project Wizard' dialog box in Xilinx ISE. The title bar reads 'New Project Wizard'. Below the title bar, the text 'Project Settings' is followed by 'Specify device and project properties.' The main area is titled 'Select the device and design flow for the project' and contains a table of properties. The properties are as follows:

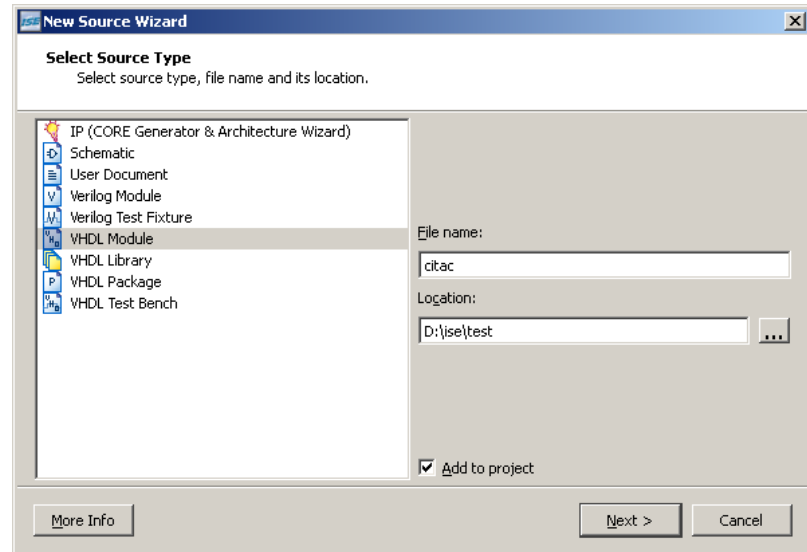
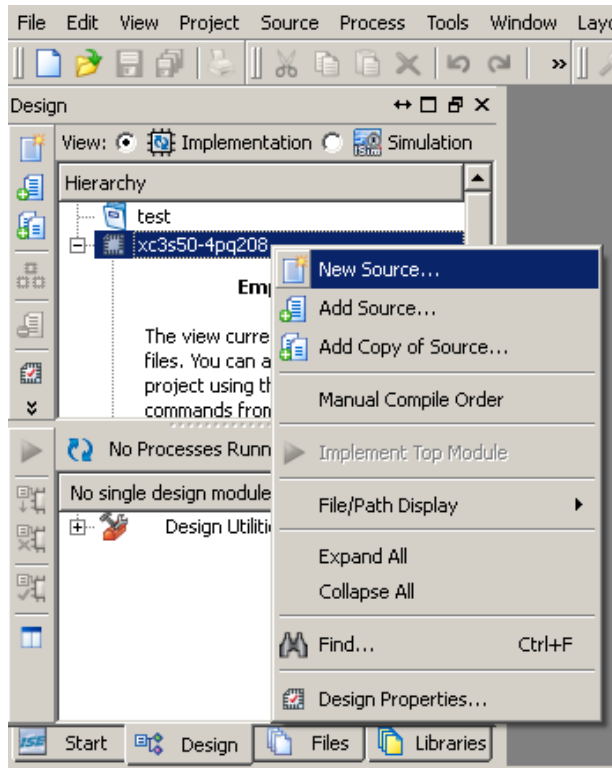
Property Name	Value
Product Category	All
Family	Spartan3
Device	XC3S50
Package	PQ208
Speed	-4
Top-Level Source Type	HDL
Synthesis Tool	XST (VHDL/Verilog)
Simulator	ISim (VHDL/Verilog)
Preferred Language	VHDL
Property Specification in Project File	Store all values
Manual Compile Order	☐
VHDL Source Analysis Standard	VHDL-93
Enable Message Filtering	☐

At the bottom of the dialog box, there are three buttons: 'More Info', '< Back', and 'Next >', and a 'Cancel' button on the far right.

Xilinx ISIM tutorial

přidání zdrojových kódů

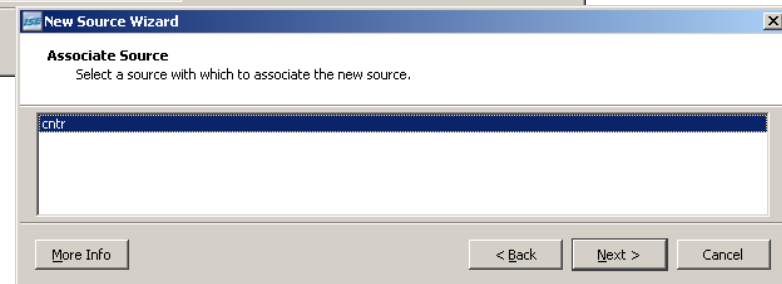
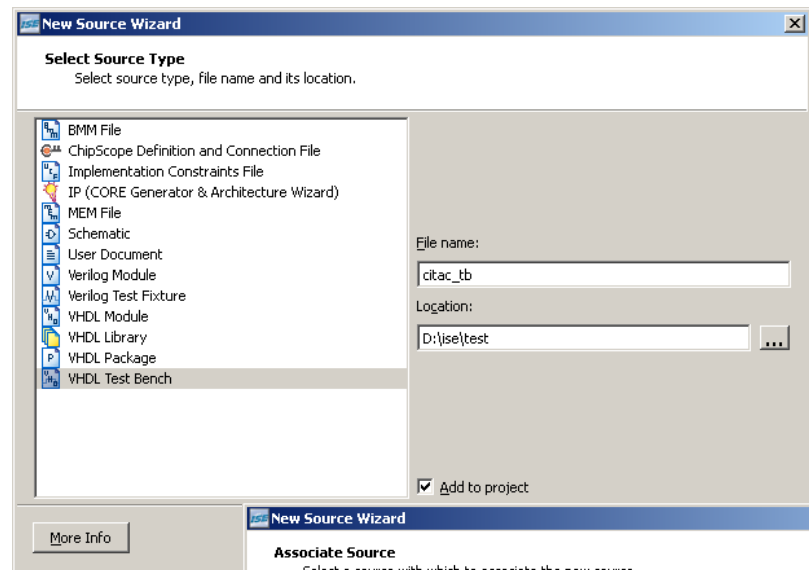
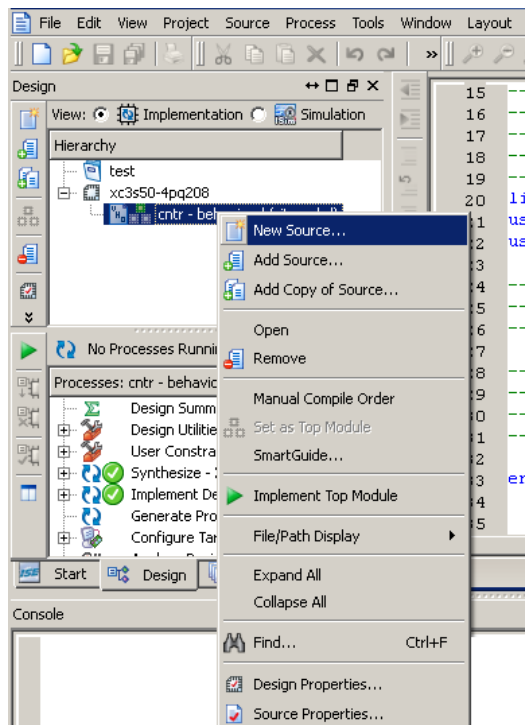
- Vytvoření zdrojového souboru s popisem čítače do projektu
 - kontextové menu *Hierarchy* > *New Source*
 - zvolit položku *VHDL Module*



Xilinx ISIM tutorial

vygenerování kostry testbench souboru

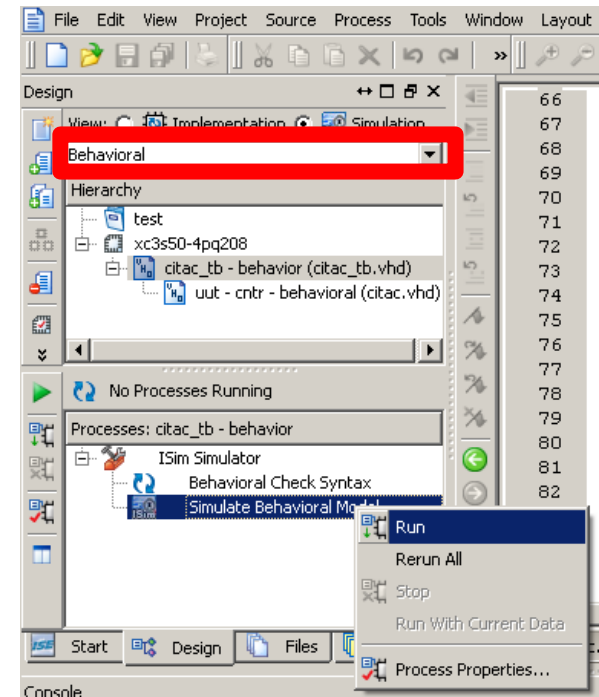
- Vytvoření zdrojového souboru obsahujícího test bench
 - kontextové menu *Hierarchy* > *New Source*
 - zvolit položku *VHDL Test Bench*
 - zvolit entitu, ke které se test bench vztahuje
 - nástroj **automaticky vygeneruje kostru kódu** pro zvolenou entitu



Xilinx ISIM tutorial

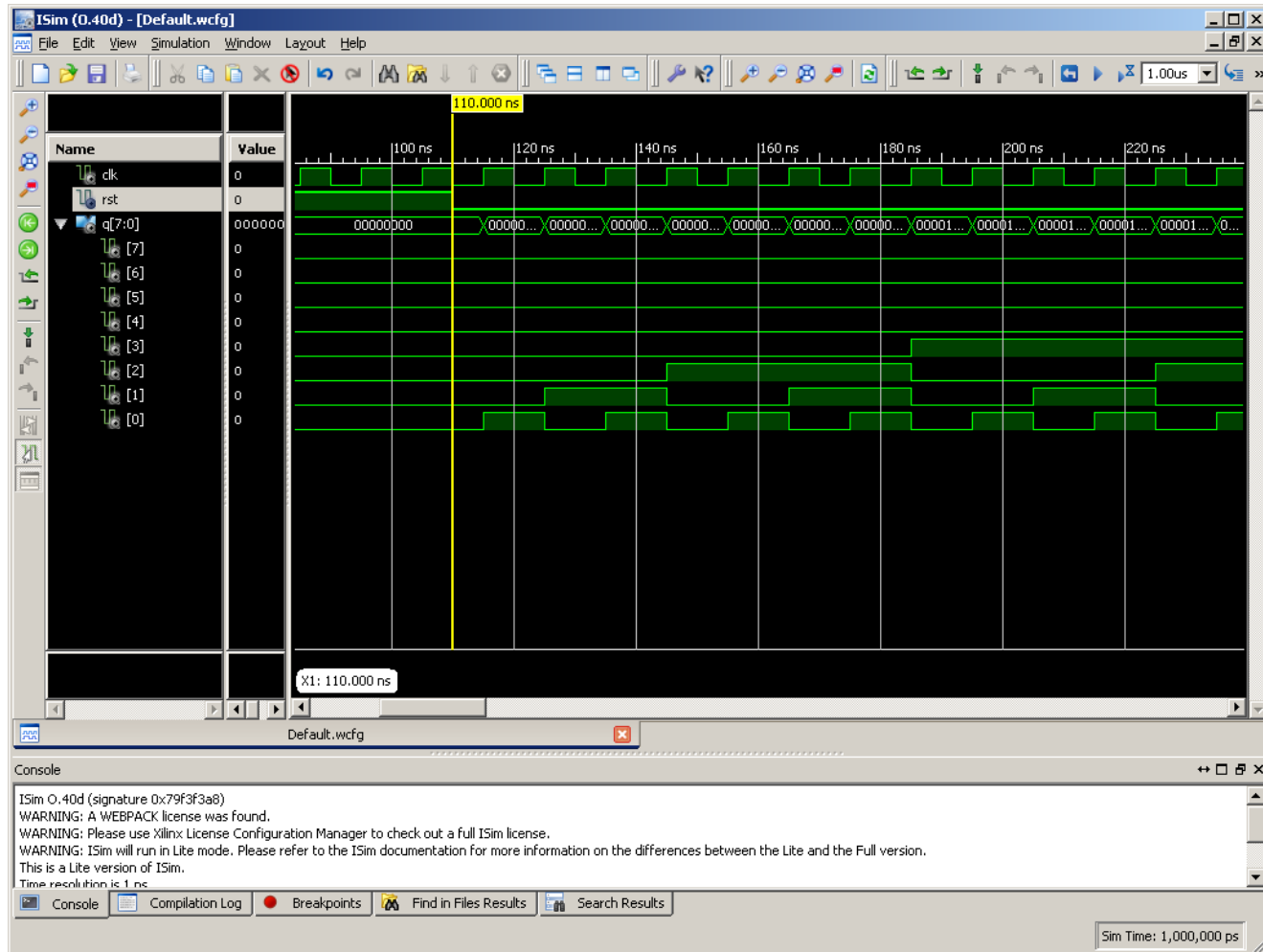
spuštění funkční simulace

- Simulace (funkční/behaviorální)
 - přepnutí pohledu na *Simulation*
 - označit položku s názvem test bench entity (*citac_tb*)
 - ve vlastnostech procesu *Simulate Behavioral Model (Process Properties)* lze nastavit délku trvání simulace
 - spustit simulaci *Processes > Simulate Behavioral Model*



Xilinx ISim tutorial

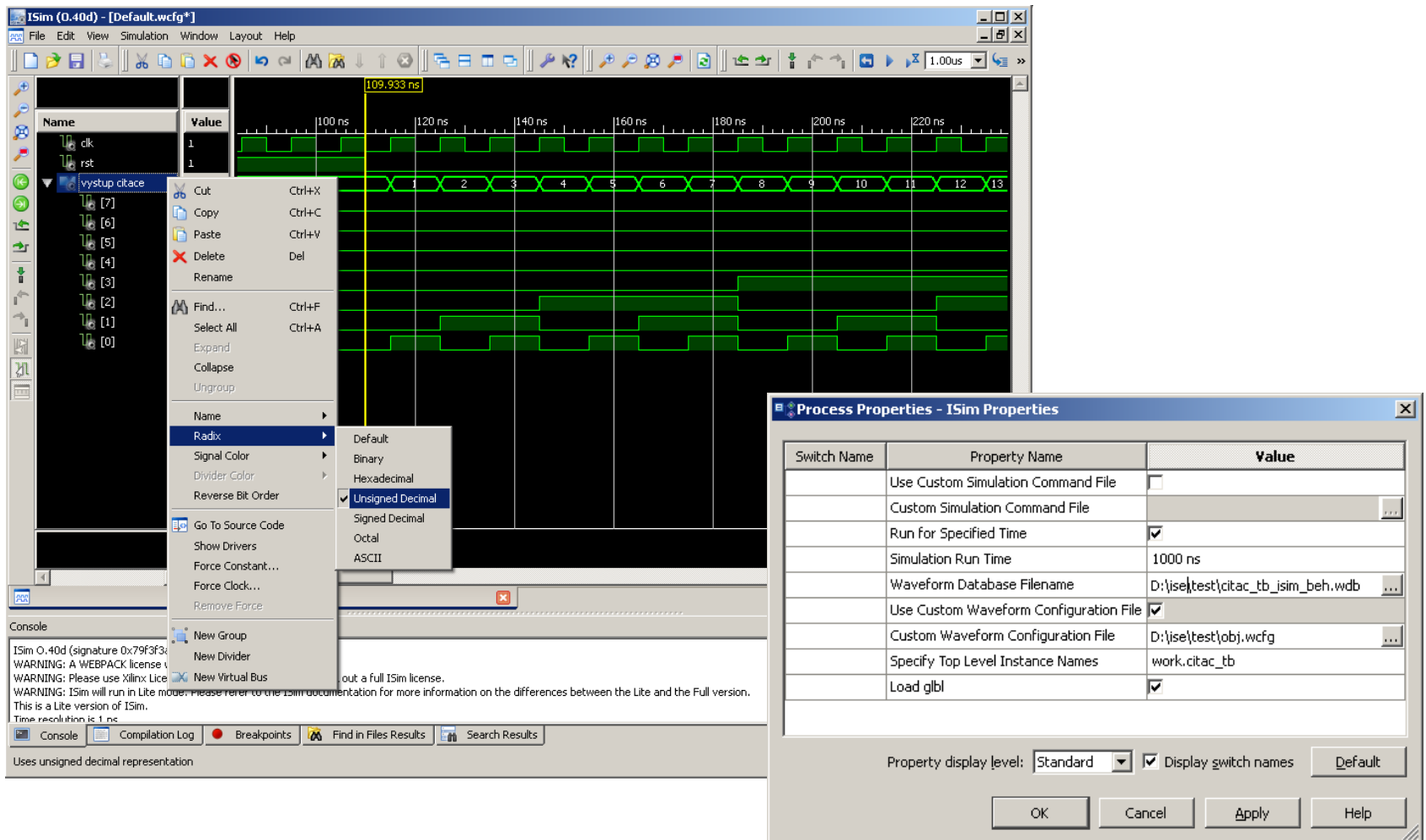
výsledek funkční simulace



V případě potřeby lze prodloužit simulaci příkazem run následovaným dobou (např. **run 1 ms**)

Xilinx ISim tutorial

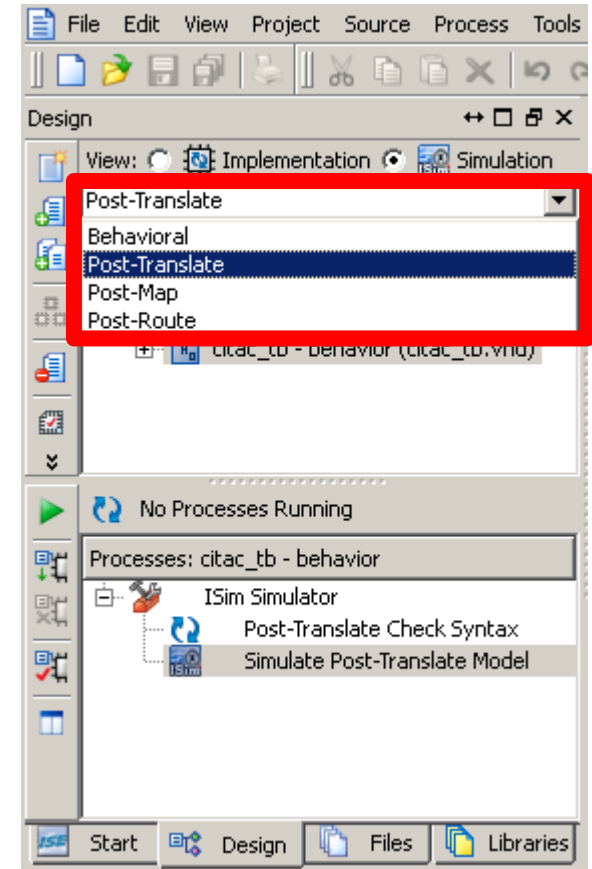
- Seznam signálů lze přizpůsobit dle potřeb, uložit (*File > Save As*) a nastavit jako výchozí pro simulaci



Xilinx ISIM tutorial

spuštění časové simulace

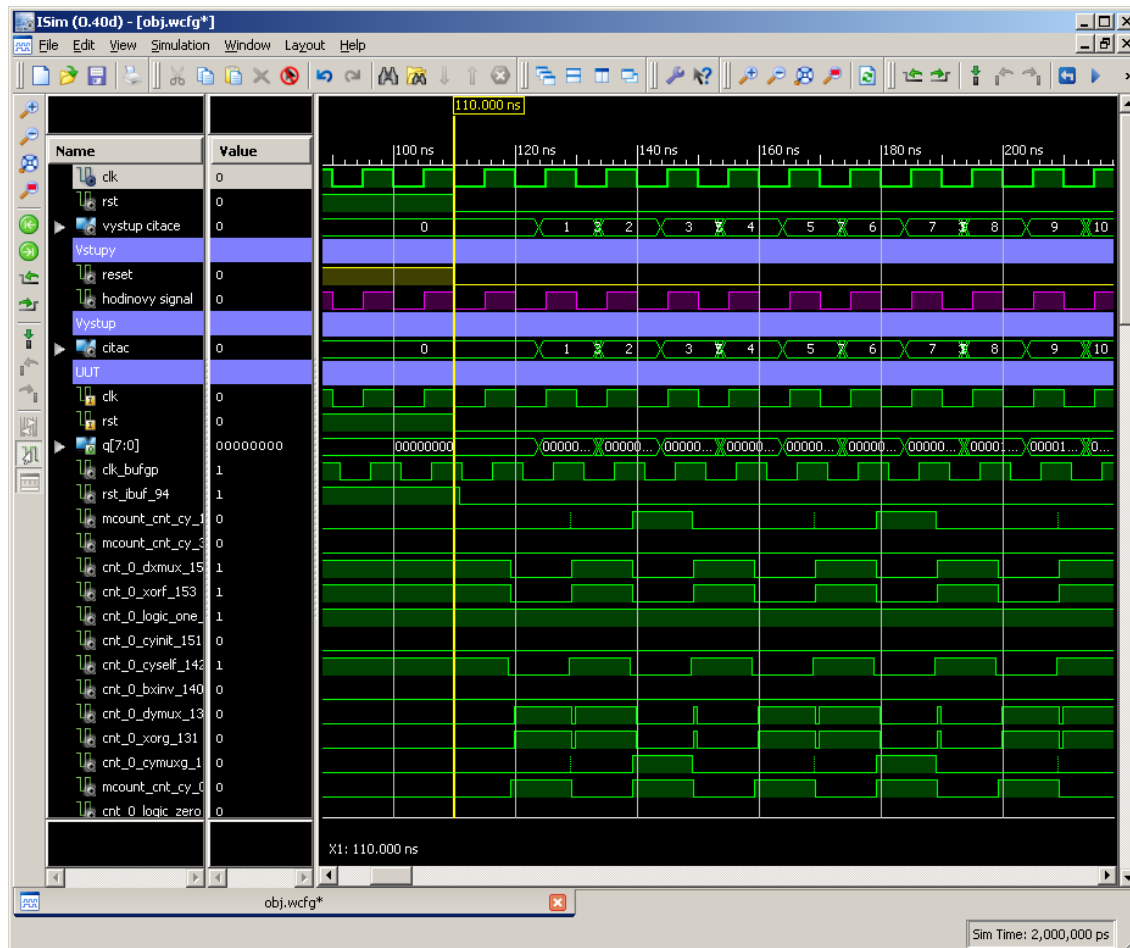
- Simulace (po syntéze, mapování)
 - přepnutí pohledu na *Simulation*
 - zvolit typ simulace
 - *Behavioral* (funkční simulace)
 - *Post-Translate*
 - *Post-Map*
 - *Post-Route* (časová simulace)
 - označit položku s názvem test bench entity (*citac_tb*)
 - spustit simulaci, např. *Processes > Simulate Post-Translate Model*



Xilinx ISIM tutorial

výsledek časové simulace

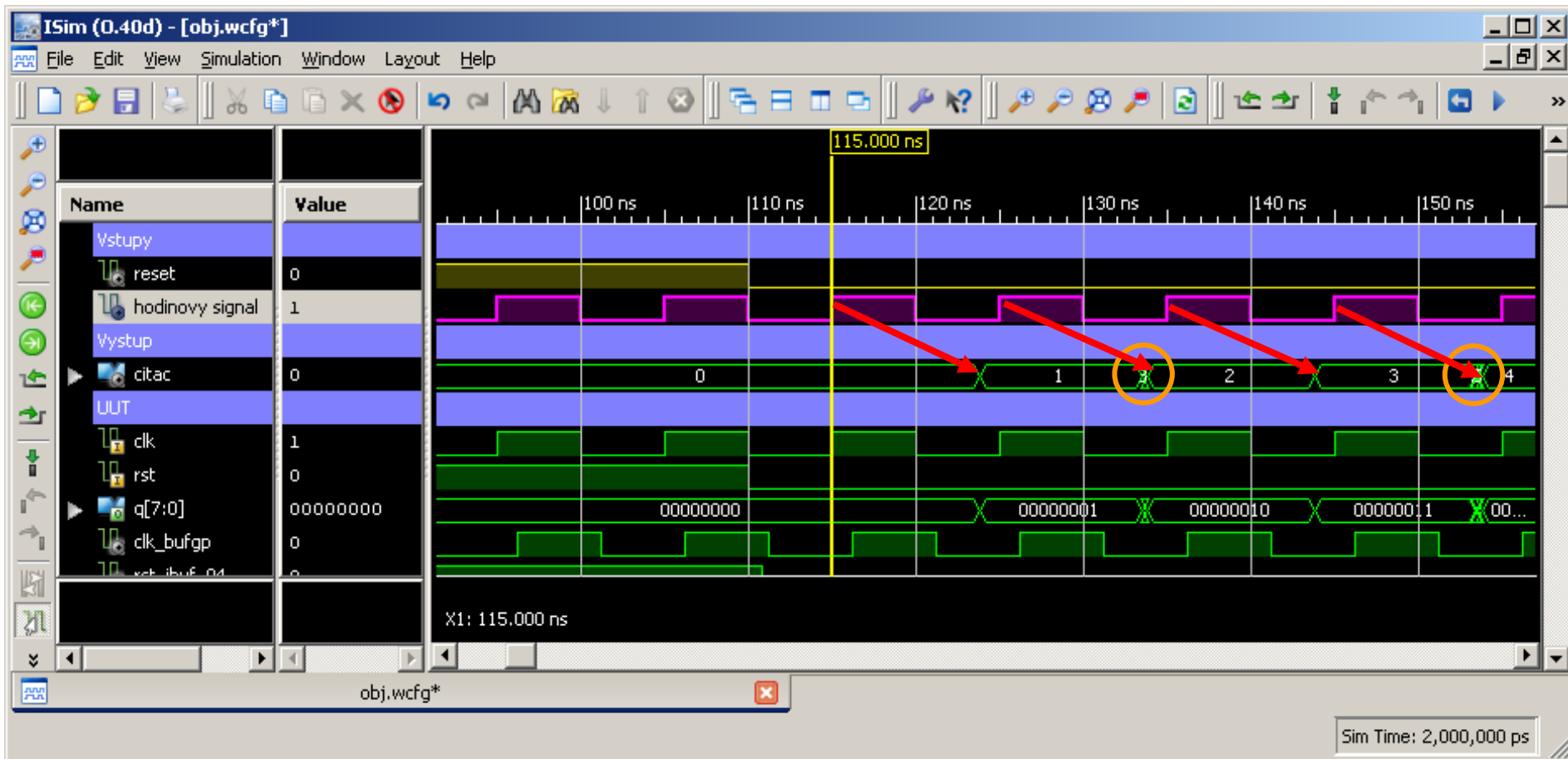
- Výsledek *Post-Route* simulace zohledňuje zpoždění komponent a vodičů (proto se nazývá časová simulace)



Xilinx ISim tutorial

výsledek časové simulace (detail)

- Ke změně výstupu v praxi nikdy nedochází ihned po náběžné hraně, syntéza však musí garantovat, že ke změně dojde vždy před novou náběžnou hranou (viz maximální pracovní frekvence)



Xilinx ISIM tutorial

chybná konstrukce registru ve funkční a časové simulaci

Rozdíl funkční a časové simulace
(chybně popsany čítač zneužívající chování sensitivity listu)

```
citac: process (RST, CLK)
begin
    if RST='1' then
        Q <= (others => '0');
    elsif (CLK = '1') then
        Q <= Q + 1;
    end if;
end process;
```

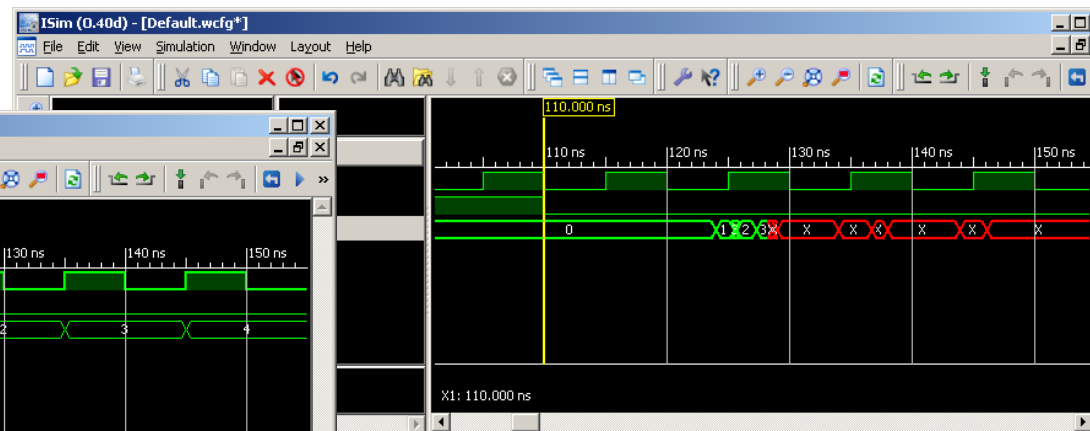
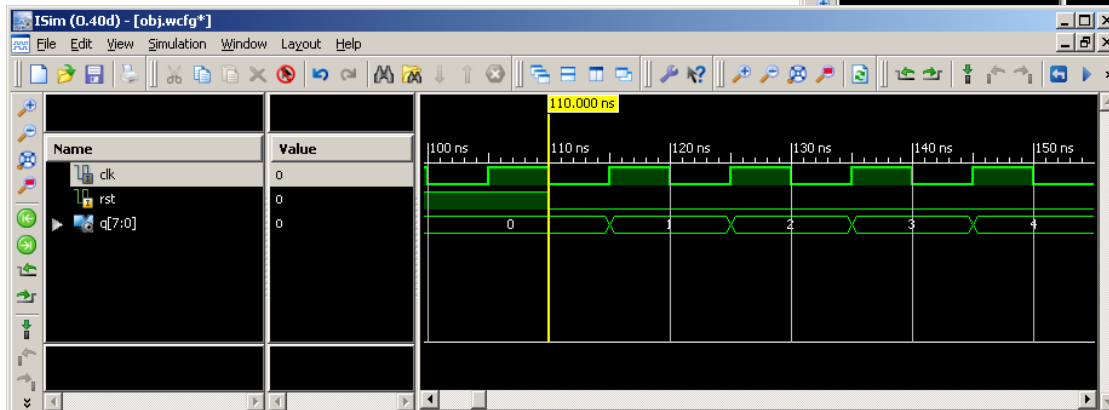
chybně (latch)

správně (registr)

```
elsif CLK'event and (CLK = '1') then
```

Behavioral Simulation

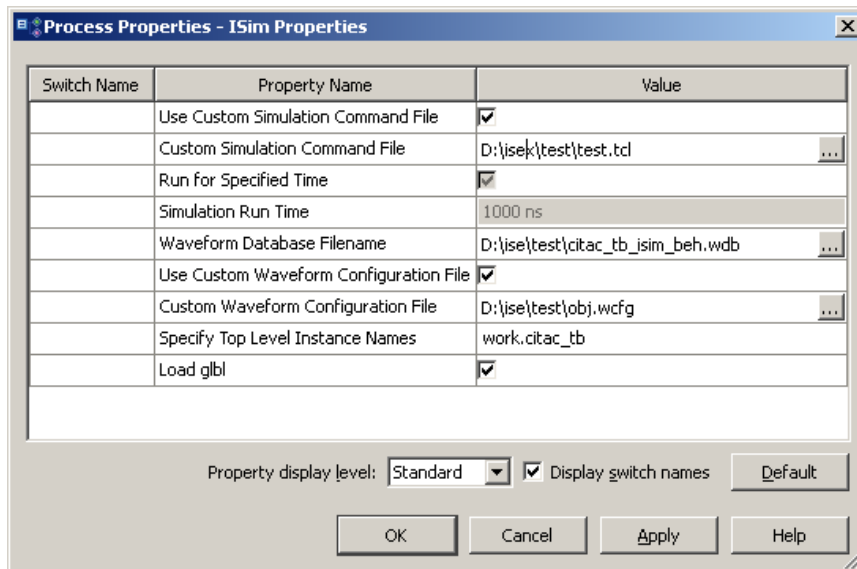
Post-Route Simulation



Xilinx ISIM tutorial

automatizace

- Uživatelské skripty (automatizace simulace)
 - ve vlastnostech procesu *Simulate Behavioral Model (Process Properties)* zvolit *Use Custom Simulation Command File* a vyplnit cestu k souboru obsahující simulační skript (kód v TCL)



Xilinx ISIM tutorial

automatizace

- Příklad skriptu

```
#zakladni signaly
divider add "Vstup"
wave add /rst -name "reset" -color yellow
wave add /clk -name "hodinovy signal" -color magenta

divider add "Vystup"
wave add /q -radix unsigned -name "citac"

#vsechny signaly z UUT
divider add "UUT"
wave add /citac_tb/uut/

isim force add {/citac_tb/rst} 1 -radix bin -time 600 ns -cancel 1000 ns
run 2us
```

- Přehled podporovaných příkazů viz
http://www.xilinx.com/tools/feature/isim_qrg.pdf

Xilinx ISim tutorial

automatizace

- Výstup z předchozího skriptu

